

Individual Game Submission
CSC 481/581 - Game Engine Foundations - Fall 2026

Group 4: Yeva Mheryan

Individual Game Name: “Hello Kitty Adventure”

GitHub Game Repository:

<https://github.com/CodeRockerr/CSC481-581-Group4/tree/main/Games/ymherya>

(A) Introduction

For Milestone 2, my main goal was to add multiplayer networking to my Hello Kitty Adventure game. My main contributions for this milestone were implementing the server and the NetworkClient. The purpose of these systems was to allow multiple instances of the game to connect to the same server and share information about the players and the moving platform. This allows me to turn my single-player game from Milestone 1 into a multiplayer game, allowing multiple players to see each other while keeping each client's local timeline independent.

(B) Design and Architecture

The client-server architecture was implemented in our team using ZeroMQ library for communication. The server operates as a headless separate process whereas all instances of Hello Kitty Adventure operate as clients. I decided to use the separate server from the Engine and SDL because the server is responsible only for managing the shared game state and does not need any rendering capabilities. That means the server can run in a terminal window without any dependence on the game window. Upon connecting to the server, it gives each client the unique client ID and communication port. Then the client sends its player state to the server and receives the information about all other connected players and about the server-driven moving platform. In order to guarantee that the data being transmitted in the network were consistent I used the shared NetMessage structure containing such data as the message type, client ID, position, velocity, and timeline. Furthermore, I implemented the NetworkClient which does not interrupt the game's work by waiting for the network communication process and performs network actions simultaneously with the rest of the game code.

Figure 1. The server running separately from the game clients.

```
$ ./build/Debug/ymherya_server.exe 5555
Starting Hello Kitty Adventure server on port 5555...
Server listening on port 5555
Client 1 connected
Client 2 connected
```

(C) Why We Chose This Design

Another crucial design choice was assigning the responsibility for maintaining the shared game state to the server rather than letting each client perform all calculations individually. This proved particularly helpful in case of Kuromi and the moving platform. The server calculates the position of Kuromi based on the WorldTime timeline and passes this state to the clients. As a result, each client receives the same position of the platform rather than calculating it separately. This makes the synchronization of the shared part of the game consistent across clients. We have used different timelines for the local player and the shared world. Hello Kitty is controlled by the local PlayerTime timeline while Kuromi is controlled by the WorldTime maintained by the server. It was essential to separate timelines since changing the timeline of one player must not affect the timeline of the other player. For instance, if one player paused the game, the other clients and the shared world controlled by the server could continue working. One more design choice that was crucial for meeting assignment requirements was using a separate NetworkClient thread. In case of waiting for every network message on the main thread, slow network connection could affect input,

Figure 2. Hello Kitty Adventure running with multiple clients connected.

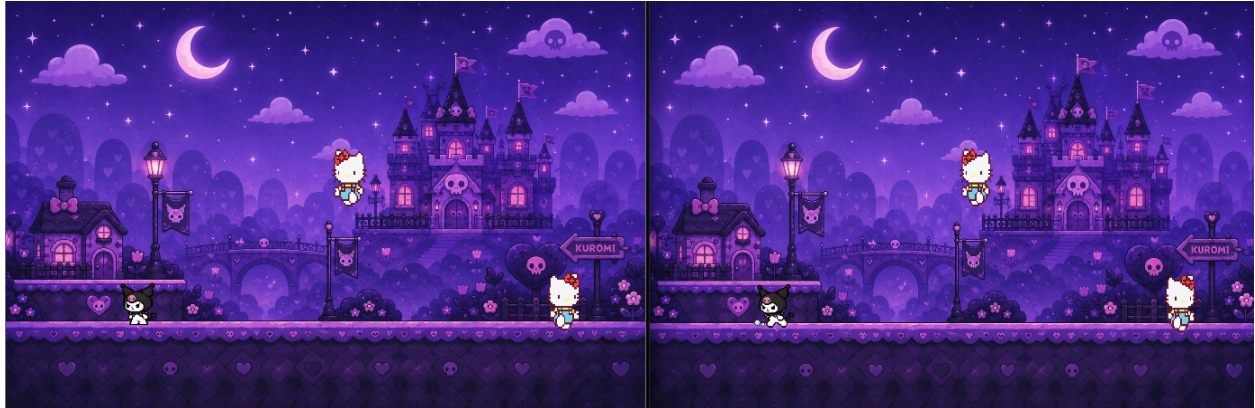
<pre>\$./build/Debug/ymherya_game.exe --client 127.0.0.1 5555 Connected to server as client 1 Hello Kitty was killed by Kuromi! Deaths: 1 █</pre>	<pre>\$./build/Debug/ymherya_game.exe --client 127.0.0.1 5555 Connected to server as client 2 Hello Kitty was killed by Kuromi! Deaths: 1 Hello Kitty was killed by Kuromi! Deaths: 2 █</pre>
--	--

(D) What the Design Enabled

The end result design allowed multiple clients to connect to the same server and see each other's player status. It also made it possible for the server to control the moving platform so that all clients will get the same world status. As seen in Figure 2 and Figure 1, it made it possible for multiple clients to be active at once in communicating with the same server. Timeline design also made it possible for the player to control their own game speed. P key will pause/unpause the local player's timeline, while keys 1, 2 and 3 will change the player's timeline to 0.5x, 1.0x and 2.0x respectively. It does not affect the other clients' timelines. One of the useful parts of this

design is that the clients do not have to share memory with each other. Instead they use the server as an interface for communication. That way the game processes become independent from one another.

Figure 3. Two Hello Kitty Adventure clients connected to the same server and displaying each other's players.



(E) What Worked and What Did Not

Almost all of the networking features were functional as planned. I tested out different clients logging into the same server, different client IDs, players being able to see other players' movements, and Kuromi synchronization across clients. I also tested the ability to change a single player's timeline without interfering with other clients. The server was capable of functioning even while clients were connecting and disconnecting. One drawback of the current implementation is that the remote players rely on the most recent position sent by the server. Therefore, there is no smoothing of the movements, which means that the movements of remote players will not be as smooth as those of the local player's. Another drawback is that the disconnected players are not deleted from the list of entities. In a future version, when the server detects that the client has disconnected, the remote player can be deactivated or even deleted.

(F) Future Improvements

The current client-server architecture provides the game with a basis which may be built on in the future. Further objects of the game could be incorporated into the network packets and server snapshot without having to alter the client-server architecture in any way. A remote movement smoothing feature could be added as well which would make the movements smoother between updates from the server. The client side prediction could be another addition to the game which could improve the responsiveness of the movement of the player due to the network lag. Further objects could also be managed by the server such as additional moving platforms, enemies or

other multiplayer actions. Since the server is not tied to the rendering process of the game using the SDL window, this could be done without any problems.

(G) Conclusion

In terms of contribution to Milestone 2, my efforts included the development of the server and the NetworkClient for Hello Kitty Adventure. Through the client-server architecture, I was able to have multiple players connected to one game world, and in addition, I maintained the responsibility of the server for the state of the game. The use of different timelines ensured that individual speed of the game per player could be adjusted separately, and at the same time, the WorldTime of the server maintained Kuromi's consistency across all the clients. Though my implementation proved effective in achieving the main aspects of multiplayer, there were aspects where improvements could still be made, particularly the movement smoothing for remote players and handling of disconnected players.

(H) Build and Run Instructions

From the project root, I build the project using:

```
Shell
cmake -S . -B build
cmake --build build
```

I then start the server in one terminal:

```
Shell
./build/Debug/ymherya_server.exe 5555
```

The server then stays in the terminal and doesn't open a game window.

I start a game client from another terminal using:

```
Shell
./build/Debug/ymherya_game.exe --client 127.0.0.1 5555
```

Additional clients can be started from separate terminals using the same command.

To stop the server, I use Ctrl + C in the terminal running the server. If needed, the server can also be stopped by using:

```
Shell
taskkill //F //IM ymherya_server.exe
```